

Documentación

Nombre: Gabriel Alexander Calderón Monzón

Carnet: 2022327

Fecha: 6/2/2026

1. Introducción:

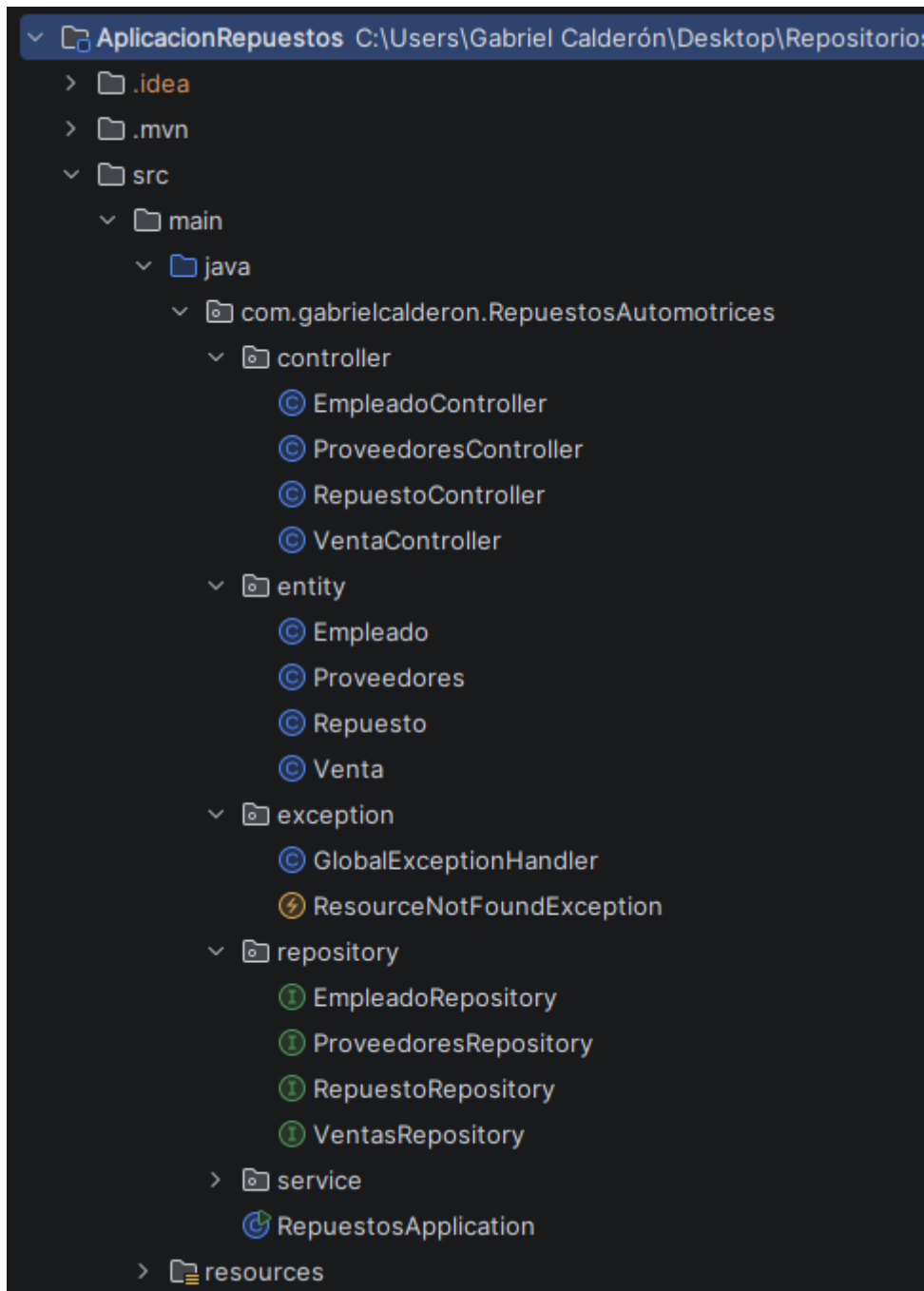
Este proyecto trata sobre una aplicación web (solo backend) de un sistema que permita que permita gestionar una tienda de repuestos automotrices, ya sea gestionar empleados, proveedores, ventas y repuestos. Permitiendo: Agregar, editar, eliminar, listar y buscar.

Este proyecto se trabajó por capas para separar funciones y que todo este en módulos, permitiendo un código ordenado, fácil de entender y de mantener.

2. Estructura del proyecto:

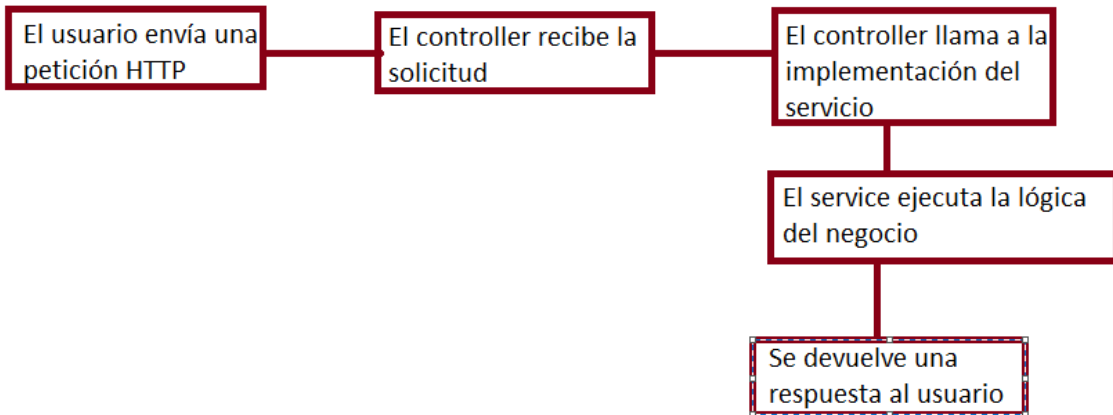
El proyecto se trabajo por capas donde cada una cumple con ciertas funciones:

- **Controller:** Gestiona las peticiones HTTP mediante endpoints. Se usan las anotaciones `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping` y `@DeleteMapping`.
- **Entity:** Mapea las entidades de la base de datos. Usando la anotación `@Entity`, además se agregan validaciones internas para los atributos, las cuales serán captadas por el `GlobalExceptionHandler`.
- **Exception:** Permite manejar excepciones de manera global y crear excepciones personalizadas.
- **Repository:** Permite la comunicación con la base de datos, mediante la dirección dada en las configuraciones. Se extiende de `JpaRepository` y hace automáticamente
- **Service:** Maneja toda la lógica del negocio desde los procesos CRUD hasta validaciones propias. Aquí se definen los métodos que se van a utilizar.



3. Funcionamiento general del sistema:

El flujo de la aplicación funciona así:

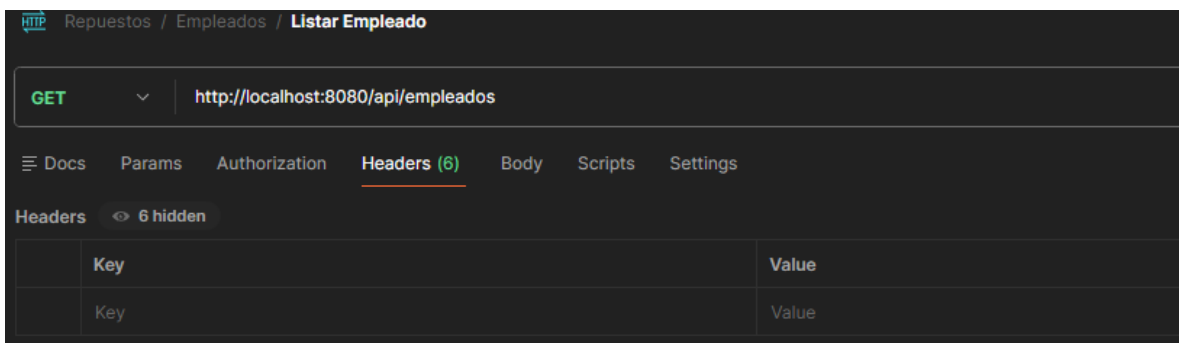


4. Métodos HTTP en Postman:

Realicé los siguientes métodos HTTP en postman:

- **POST:** Agrega información.
- **GET:** Obtiene información.
- **PUT:** Actualiza información.
- **DELETE:** Eliminar información.

Todo se trabaja mediante endpoints, también conocidos como rutas que fueron definidas en los controllers, lo dividí las carpetas por entidades para llevar un mejor orden.



POST ▼ <http://localhost:8080/api/empleados>

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "nombre_empleado" : "Gabriel",
3   "apellido_empleado" : "Calderon",
4   "puesto_empleado" : "Gerente",
5   "email_empleado" : "gcalderon@gmail.com"
6 }
```

[HTTP](#) Repuestos / Empleados / **Actualizar Empleado**

PUT ▼ <http://localhost:8080/api/empleados/1>

Docs Params Authorization Headers (8) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "nombre_empleado" : "Alexander",
3   "apellido_empleado" : "Monzon",
4   "puesto_empleado" : "Desarrollador",
5   "email_empleado" : "amonzon@gmail.com"
6 }
```

[HTTP](#) Repuestos / Empleados / **Buscar Empleado**

GET ▼ <http://localhost:8080/api/empleados/11>

Docs Params Authorization Headers (6) **Body** Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

DELETE ▼ <http://localhost:8080/api/empleados/8>

Docs Params Authorization Headers (6) **Body** Scripts Settings

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL